45. Numerics and Tensors in Go (gonum)

45. Numerics and Tensors in Go (gonum)

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `45-numerics-gonum`

Learning objectives

- Use float64 carefully
- Perform vector ops with gonum-style APIs
- Know floating-point pitfalls
- Benchmark numeric kernels

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Numerics with gonum enable scientific and ML-adjacent work in Go: vectors, matrices, and stats without leaving the language.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Vectors/matrices are first-class for linear algebra.
- Dot products and norms appear in similarity search.
- Floating error: use tolerances in tests.
- Prefer well-tested libraries over hand-rolled BLAS.
- Shape mismatches should error early.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-numerics && cd /tmp/gw-numerics
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

func Dot(a, b []float64) float64 {
    var s float64
    for i := range a {
        s += a[i] * b[i]
    }
    return s
}

func main() { fmt.Println(Dot([]float64{1, 2}, []float64{3, 4})) }
```

Run it:

```
go run .
```

Expected output:

```
11
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Implement Dot and L2 norm.
2. Test with an epsilon comparison.
3. Multiply a tiny matrix by a vector by hand in Go.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Floating point

Floats are approximations. Never compare with == without tolerance. Summation order changes results; catastrophic cancellation is real.

gonum

The gonum organization provides matrices, stats, graph, optimize packages — Go's scientific backbone.

```
// Dot product
func Dot(a, b []float64) float64 {
    var s float64
    for i := range a { s += a[i] * b[i] }
    return s
}
```

AI relevance

Even when models run elsewhere, Go services still normalize features, compute similarities, and batch tensors as bytes.

Failure modes to watch while coding

- Equality checks on floats
- Ignoring NaN propagation
- Using float32 without need then losing precision

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- numpy is richer; Go/gonum is explicit and deployable in one binary.
- Broadcasting magic vs explicit loops — clarity wins in Go.
- Same floating-point caveats.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

For heavy training, Python/CUDA still dominates; Go shines at serving and glue.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Numerics and Tensors in Go (gonum)” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Equality checks on floats
- Ignoring NaN propagation

- Using float32 without need then losing precision

Exercises

Work in `exercises/45-numerics-gonum/`. Read that folder's `README.md` before you code.

What you will build

Implement Dot (or gonum-backed stub) matching tests.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/45-numerics-gonum/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Why epsilon comparisons?
2. What is gonum used for?
3. Dot product complexity?

Further reading

- https://www.gonum.org/